BIZEV-01: Business Events Fundamentals

> **Series:** BIZEV — Business Events & Funnel Analysis | **Notebook:** 1 of 7 | **Created:** March 2026 | **Last Updated:** 10/25/2026

Overview

Business events are purpose-built telemetry that captures meaningful business transactions — purchases, logins, form submissions, API calls — as first-class observability data in Dynatrace Grail. Unlike generic events or logs that focus on infrastructure health, business events (`bizevents`) tie technical signals directly to business outcomes. This notebook introduces the business events data model, explains how they differ from generic events, and demonstrates the core DQL patterns for exploring business event data.

Primary Fields on Business Events (OneAgent 1.333+)

OneAgent can stamp **primary Grail fields and primary tags** onto the telemetry it sends — metrics, spans, logs, events and entities — and Dynatrace's own example queries business events by them (`fetch bizevents | filter dt.cost.costcenter == "payments"`). For BIZEV authors this means:

- **`dt.security_context`** rides on business events from OneAgent-instrumented sources — useful when business events carry data subject to regulatory boundaries (PCI, GDPR, SOX). Record-level permissions can be conditioned on this field directly without OpenPipeline parsing.
- **`dt.cost.costcenter`** / **`dt.cost.product`** and customer-defined `primary_tags.*` ride along too — funnel and conversion dashboards can segment by cost center or product line without per-event tagging in application code.
- For business events sent through the HTTP API or the RUM/mobile APIs (not OneAgent-captured), add equivalent fields at ingest with a processor in the OpenPipeline Processing stage.

> **Sources:** Primary Grail fields and tags enrichment through OneAgent (DT docs) — *"OneAgent can enrich telemetry data at the source with custom metadata before sending metrics, spans, logs, events, and entities."*

Table of Contents

1. What Are Business Events?
2. Business Events vs Generic Events
3. The Bizevents Data Model
4. Ingestion Methods
5. Exploring Business Events with DQL
6. Event Providers and Categories
7. Summary and Next Steps

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
Permissions	`storage:events:read`, `storage:bizevents:read`
Data	Business events being ingested (OneAgent auto-capture or API)
Knowledge	Basic DQL query skills

> **Running Dynatrace Managed, or a SaaS tenant still operated through the classic (Gen2) experience?** Business events require Grail and are SaaS-only — but a Grail-enabled SaaS tenant can adopt them today with a minimal Gen3 footprint (new Dashboards/Notebooks for this one workload, everything else stays classic), and Managed environments have documented classic approximations. See **BIZEV-**

> **07: Gen2 vs Gen3 — Business Events Without (or Before) the Full Move** before deciding this series doesn't apply to you.

1. What Are Business Events?

A **business event** represents a discrete business transaction or user action that has direct business meaning. Examples include:

- A customer completing a purchase
- A user logging into a portal
- A payment being processed
- An order being shipped
- A search query being executed

Business events are stored in the `bizevents` data object in Dynatrace Grail. They carry a structured schema that includes:

Field	Description	Example
`event.type`	The business event type identifier	`com.myapp.purchase.completed`
`event.provider`	The source system or application	`www.easytravel.com`
`event.category`	Optional significance category (ITIL event management)	`Availability`
`timestamp`	When the event occurred	`2026-03-12T10:30:00Z`

> **Note:** Business events can also carry arbitrary custom attributes as part of their payload — product IDs, order amounts, user segments, geographic data, and more.

Let's start by fetching recent business events to see what data is available in your environment.

```
// Fetch the most recent business events to explore available data
fetch bizevents, from:-24h
| sort timestamp desc
| limit 20
```

2. Business Events vs Generic Events

Dynatrace has two distinct event data objects. Understanding the difference is critical for choosing the right approach.

Aspect	Business Events (`bizevents`)	Generic Events (`events`)
Purpose	Business transactions and user actions	Infrastructure and operational signals
Schema	`event.type` , `event.provider` , custom payload	`event.kind` , `event.type` , DT-enriched
Sources	OneAgent auto-capture, API, OpenPipeline	OneAgent, Extensions, Dynatrace Intelligence
Typical use	Funnel analysis, revenue tracking, KPIs	Alerting, incident management
Retention	Configurable via Grail buckets	Configurable via Grail buckets
DQL table	`fetch bizevents`	`fetch events`

> **Important:** Do not confuse `bizevents` with `events` . They are separate data objects with different schemas and use cases. Business events are designed for business analytics; generic events are designed for operational monitoring.

```
// Compare the volume of business events vs generic events
fetch bizevents, from:-24h
| summarize bizevent_count = count()
| append [
    fetch events, from:-24h
    | summarize generic_event_count = count()
  ]
```

3. The Bizevents Data Model

Every business event in Grail follows a standard schema with required and optional fields.

Required Fields

Field	Type	Description
`event.type`	string	Unique identifier for the event type
`event.provider`	string	Source application or system
`timestamp`	timestamp	When the event occurred

Common Optional Fields

Field	Type	Description
`event.category`	string	Significance of the event per the ITIL event management standard (e.g., `Availability`)
`event.group`	string	Sub-grouping within a category
`dt.entity.service`	entity ID	Associated service entity — `deprecated`; use `dt.smartscape.service`
`dt.source_entity`	entity ID	Entity that generated the event — `deprecated`; use `dt.smartscape_source.id`

> **Dictionary:** `dt.entity.service` (`deprecated`), `dt.smartscape.service` (`stable`), `dt.source_entity` (`deprecated`), `dt.smartscape_source.id` (`stable`), `event.category` (`stable`), read 10/25/2026.

Custom Payload Fields

Business events can carry any number of custom attributes. These are typically defined during instrumentation:

```
# Example custom payload fields
order_id: "ORD-12345"
total_amount: 149.99
currency: "USD"
customer_segment: "premium"
payment_method: "credit_card"
```

```
// Explore the schema of business events — see all available fields
fetch bizevents, from:-24h
| limit 5
| fieldsKeep event.type, event.provider, event.category, timestamp
```

```
// Count distinct event types and providers to understand data diversity
fetch bizevents, from:-24h
| summarize {event_types = countDistinct(event.type),
           providers = countDistinct(event.provider),
           total_events = count()}
```

4. Ingestion Methods

Business events reach Dynatrace Grail through several ingestion pathways.

Method 1: OneAgent Auto-Capture

OneAgent can automatically capture business events from web requests when configured via **business event capture rules** in Settings. This requires:

- OneAgent deployed on the application host
- Capture rules defined in **Settings > Collect and Capture > Business events**
- Request attributes or session properties mapped to event fields

Method 2: Business Events API

Send events directly via the Dynatrace API:

```
curl -X POST "https://{your-
environment}.live.dynatrace.com/api/v2/bizevents/ingest" \
  -H "Authorization: Api-Token dt0c01.xxxxx" \
  -H "Content-Type: application/cloudevents+json" \
  -d '{
    "specversion": "1.0",
    "id": "unique-event-id",
    "type": "com.myapp.order.completed",
    "source": "myapp-backend",
    "data": {
      "order_id": "ORD-12345",
      "amount": 149.99
    }
  }'
```

> **Note:** The API expects CloudEvents format. The `type` field maps to `event.type` and `source` maps to `event.provider` in DQL.

Method 3: OpenPipeline Extraction from Logs and Spans

OpenPipeline can create business events from other telemetry: a **Business event** processor in the **Data extraction** stage of a logs or spans pipeline emits a new bizevent for each matching record. This is useful when business signals are embedded in existing telemetry.

Method 4: RUM and Mobile `sendBizEvent`

Browser code sends business events through the RUM JavaScript API; the mobile agents and OpenKit offer equivalent methods:

```
// Browser (RUM JavaScript API)
dynatrace.sendBizEvent('com.myapp.checkout.completed', {
  order_id: 'ORD-12345',
  total: 149.99,
  items: 3
});
```

There is no OneAgent SDK method for business events; backend code without a capture rule uses the API (Method 2). Workflows can also emit business events with the **Ingest business event** action.

```
// Identify ingestion sources — which providers are sending business events?
fetch bizevents, from:-24h
| summarize event_count = count(), by:{event.provider}
| sort event_count desc
| limit 10
```

5. Exploring Business Events with DQL

Now that we understand the data model, let's explore business events using core DQL patterns.

Count Events by Type

The most fundamental query — how many events of each type are being captured?

```
// Count business events by type over the last 24 hours
fetch bizevents, from:-24h
| summarize event_count = count(), by:{event.type}
| sort event_count desc
| limit 20
```

Event Volume Over Time

Visualize how business event volume changes throughout the day. This helps identify peak business hours and detect anomalies.

```
// Business event volume over time, grouped by event type
fetch bizevents, from:-24h
| makeTimeseries event_count = count(), by:{event.type}, interval:1h
```

```
// Total business event volume per hour (all types combined)
fetch bizevents, from:-24h
| makeTimeseries total_events = count(), interval:1h
```

6. Event Providers and Categories

Understanding which providers are sending events and how they are categorized helps organize your business analytics.

```
// Breakdown by provider and category
fetch bizevents, from:-24h
| summarize event_count = count(), by:{event.provider, event.category}
| sort event_count desc
| limit 20
```

```
// Business events over time, grouped by provider
fetch bizevents, from:-24h
| makeTimeseries event_count = count(), by:{event.provider}, interval:1h
```

7. Summary and Next Steps

In this notebook, you learned:

- **What business events are** and how they differ from generic events
- **The bizevents data model** — required fields (`event.type`, `event.provider`) and custom payload
- **Four ingestion methods** — OneAgent capture rules, API, OpenPipeline extraction, and the RUM/mobile APIs
- **Core DQL patterns** for exploring business events: counting, grouping, and time-series visualization

Next Steps

- **BIZEV-02: Instrumentation** — Learn how to instrument your application to capture meaningful business events
- **BIZEV-03: Funnel Analysis** — Build conversion funnels from business events

References

- Dynatrace Business Analytics
- Environment API (DT docs)

- Business event capture (DT docs) — *"Obtain RUM business events by leveraging a dedicated method of the RUM JavaScript API, OneAgent for Mobile, or OpenKit."*
- RUM JavaScript API — dynatrace (DT docs)
- CloudEvents Specification

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.